

Math Behind the Continuous JetBot Go-To Controller

June 18, 2026

1 Purpose

This note explains the control law implemented in `scripts/vicon_goto_continuous_viewer.py`. The controller uses the latest Vicon pose of the JetBot and continuously computes left and right motor commands that steer the robot toward a target point in the plane.

2 State and Target

Let the robot position in the Vicon plane be

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix},$$

and let the target position be

$$\mathbf{p}_d = \begin{bmatrix} x_d \\ y_d \end{bmatrix}.$$

The displacement to the target is

$$\Delta\mathbf{p} = \mathbf{p}_d - \mathbf{p} = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} x_d - x \\ y_d - y \end{bmatrix}.$$

The distance to the goal is

$$r = \|\Delta\mathbf{p}\|_2 = \sqrt{(\Delta x)^2 + (\Delta y)^2}.$$

This corresponds to the code lines that compute `dx`, `dy`, and `distance`.

3 Heading Estimate

The Vicon rigid body provides a full 3D orientation. The controller extracts the robot's physical forward axis from the rotated body-Y direction and then uses only its projection onto the ground plane.

Let θ denote the robot heading in the horizontal plane. The script computes

$$\theta = \text{atan2}(f_y, f_x) + \delta,$$

where:

- (f_x, f_y) is the horizontal forward direction from the Vicon pose,

- δ is the user-entered angle correction in radians.

The desired target heading is

$$\theta_d = \text{atan2}(\Delta y, \Delta x).$$

The heading error is then wrapped to $(-\pi, \pi]$:

$$e_\theta = \text{wrapToPi}(\theta_d - \theta).$$

4 Arrival Condition

If the robot is already close enough to the target, the controller stops:

$$\text{if } r \leq \varepsilon, \quad u_L = 0, \quad u_R = 0,$$

where ε is the user-selected goal tolerance.

5 Forward-Speed Schedule

The controller uses a base forward speed that depends on the target distance. Let:

- $v_{\max}$ be the nominal forward speed (`self._speed`),
- $v_{\min}$ be the minimum forward speed near the goal (`self._min_forward_speed`),
- r_s be the slow radius (`self._slow_radius`).

Then the base forward term is

$$v_{\text{base}} = \begin{cases} v_{\max}, & r \geq r_s, \\ v_{\min} + (v_{\max} - v_{\min}) \frac{r}{r_s}, & r < r_s. \end{cases}$$

So outside the slow radius the controller uses full commanded forward speed, and inside the slow radius it linearly ramps down toward $v_{\min}$.

6 Heading-Alignment Scaling

The controller reduces forward motion when the robot is pointed away from the target. It computes

$$a = \max(0, \cos e_\theta).$$

Then the actual forward command becomes

$$v = v_{\text{base}} a^2.$$

This has a useful effect:

- if $e_\theta \approx 0$, then $\cos e_\theta \approx 1$ and v stays large,
- if the robot is moderately misaligned, v is reduced,
- if the robot points more than 90° away from the target, then $\cos e_\theta < 0$, so $a = 0$ and the forward term becomes zero.

7 Steering Term

The steering command is proportional to heading error:

$$\omega = \text{sat}(k_{\theta}e_{\theta}, -u_{\text{turn}}, u_{\text{turn}}),$$

where:

- k_{θ} is the heading gain (`self.heading_gain`),
- u_{turn} is the turn-speed limit (`self.turn_speed`),
- $\text{sat}(x, a, b)$ clamps x into the interval $[a, b]$.

8 Differential-Drive Motor Commands

Finally, the left and right motor commands are

$$u_L = \text{sat}(v - \omega, -1, 1), \quad u_R = \text{sat}(v + \omega, -1, 1).$$

This is the core differential-drive mixing rule:

- the shared term v pushes both wheels forward,
- the signed steering term ω speeds one wheel up and slows the other down,
- clamping keeps the commands inside the valid motor range.

9 Controller Interpretation

Putting everything together, each control cycle does the following:

1. read the latest Vicon pose,
2. compute the target displacement and distance,
3. compute current heading and heading error,
4. stop if the robot is inside the goal tolerance,
5. choose a base forward speed from the distance,
6. attenuate forward motion if the heading is poor,
7. compute a bounded steering correction,
8. convert forward plus steering into left and right wheel speeds.

In compact form,

$$\begin{aligned}
r &= \sqrt{(\Delta x)^2 + (\Delta y)^2}, \\
\theta_d &= \text{atan2}(\Delta y, \Delta x), \\
e_\theta &= \text{wrapToPi}(\theta_d - \theta), \\
v_{\text{base}} &= \begin{cases} v_{\text{max}}, & r \geq r_s, \\ v_{\text{min}} + (v_{\text{max}} - v_{\text{min}}) \frac{r}{r_s}, & r < r_s, \end{cases} \\
v &= v_{\text{base}} \max(0, \cos e_\theta)^2, \\
\omega &= \text{sat}(k_\theta e_\theta, -u_{\text{turn}}, u_{\text{turn}}), \\
u_L &= \text{sat}(v - \omega, -1, 1), \\
u_R &= \text{sat}(v + \omega, -1, 1).
\end{aligned}$$

10 Connection to the Code

The most relevant code block is in `ContinuousGoToController.run()`, especially the section around:

- pose and distance computation,
- heading and heading-error computation,
- slow-radius logic,
- forward/steering mixing,
- left/right command output.